2026-11-06 글 알림

- 날짜: 2026-11-06
- 앱: `lgcns-fe-next`
- 기준 코드: `prisma/schema.prisma`의 `Notification`, `service/postService.ts`의 `notifyAuthor`, `service/notificationService.ts`, `actions/notificationActions.ts`, `component/organisms/NotificationMenu.tsx`, `component/templates/Header.tsx`
- 이 문서는 그 파일이 지금 디스크에 있는 동작만 적는다. 푸시 알림, 이메일, 웹소켓은 이 경로에 없다.

다른 사람이 내 글에 좋아요를 누르거나 댓글을 달면 `Notification` 행이 생긴다. 헤더의 `NotificationMenu`는 그 목록을 5초마다 다시 읽고, 화면을 연 뒤에 새로 도착한 읽지 않은 알림만 토스트로 그린다.

1. 파일이 언제 실행되는가

`app/layout.tsx`에는 알림 코드를 불러오는 `<script>`가 없다. `async`, `defer`, `type="module"` 속성을 단 스크립트 태그도 없다. 같은 레이아웃의 `ServiceWorkerRegister`는 프로덕션에서만 `/sw.js`를 등록한다. `NotificationMenu`는 그 등록을 호출하지 않는다.

파일	선언	실행 시점	async / defer / type="module"과의 차이
`prisma/schema.prisma`	`model Notification`	Prisma가 스키마로 클라이언트를 만들 때 쓰인다. 브라우저가 이 파일을 실행하지 않는다	스크립트가 아니다
`service/postService.ts`	`async function notifyAuthor`. export는 `postService` 객체다. 이 함수 자체는 export되지 않는다	서버에서 `toggleLike`이 좋아요를 새로 만들 때, `createComment`가 댓글을 만든 뒤에 호출된다	함수의 `async`는 ECMAScript 비동기 함수다. HTML `async` 속성이 아니다
`service/notificationService.ts`	`async function listForUser`,	서버 액션이 호출할 때만	브라우저 모듈 그래프에 들어가지

파일	선언	실행 시점	async / defer / type="module"과의 차이
	markRead, markAllRead. export const notificationService	Prisma를 실행한다	않는다
actions/notificationActions.ts	파일 첫 줄 "use server". export async function listNotifications	클라이언트 메뉴가 이 함수를 호출할 때 서버에서 실행된다. 페이지를 열 때 버튼 핸들러로 실행되지 않는다	"use server"는 번들러 지시문이다. <script>의 async가 아니다
component/organisms/NotificationMenu.tsx	파일 첫 줄 "use client". export default function NotificationMenu	서버가 헤더 HTML을 만든 뒤, 브라우저가 클라이언트 번들을 받아 hydrate할 때 함수가 다시 실행된다. 그 다음 useEffect가 setInterval로 5초마다 목록을 요청한다	"use client"는 <script type="module">이 아니다. 이 파일 경로를 src로 받는 태그는 없다
component/templates/Header.tsx	export default function Header. "use client" 없음	서버 컴포넌트로 헤더를 그린다. NotificationMenu를 자식으로 둔다	서버 컴포넌트다. 브라우저가 이 파일을 defer로 실행하지 않는다

HTML에서 async 스크립트는 받는 즉시 실행하고 문서 파싱을 막지 않으며 순서가 없다. defer는 파싱을 막지 않고, 문서 파싱이 끝난 뒤 적힌 순서대로 실행한다. type="module"은 그 지연 실행에 import/export가 더해진다. 이 알림 경로는 그 세 속성으로 붙지 않는다. 행을 만드는 코드는 서버 요청 안에서 실행되고, 배지와 토스트는 hydrate된 뒤의 setInterval 콜백에서 갱신된다.

setInterval은 브라우저 타이머다. 스크립트 로드 순서와 무관하다. 탭이 숨겨지면 document.visibilityState가 "visible"이 아니므로 load는 반환하고 간격도 다시 걸지 않는다. 탭이 다시 보이면 한 번 읽고 간격을 다시 건다.

참고: HTML script 요소, MDN script, ECMA-262 비동기 함수, MDN setInterval, MDN visibilitychange, Next.js 서버 컴포넌트와 클라이언트 컴포넌트.

2. 행이 생기는 조건

notifyAuthor는 authorId와 actorId가 같으면 바로 반환한다. 내 글에 내가 누른 좋아요와 내가 단 댓글은 알림이 되지 않는다.

좋아요는 toggleLike이 기존 행이 없을 때만 만든다. 글의 authorId를 찾은 뒤에 kind: "like"로 notification.create를 한다. 이미 있으면 좋아요를 지우고, 같은 actorId, postId, kind: "like" 알림을 deleteMany로 지운다. 댓글은 comment.create 다음에 kind: "comment"로 만든다. 글을 삭제하면 그 postId의 알림도 deleteMany로 지운다.

readAt은 만들 때 넣지 않는다. 그래서 새 행은 읽지 않은 상태다.

이 함수는 서버 액션 actions/postActions.ts의 toggleLike, createComment와 app/api/posts/[postId]/like, comments 라우트가 postService를 호출할 때 같이 실행된다. 알림 전용 라우트는 없다.

3. 목록과 배지

listNotifications는 readCurrentUser가 없으면 { items: [], unreadCount: 0 }을 반환한다. 있으면 notificationService.listForUser를 호출한다.

listForUser는 Promise.all로 두 쿼리를 같이 보낸다. findMany는 recipientId가 현재 사용자인 행을 createdAt 내림차순으로 20개만 가져온다. count는 그 사용자의 readAt이 null인 행 전체다. 배지 숫자는 화면에 나열된 20개가 아니라 이 count다. 20개를 넘으면 버튼에는 9+로 줄인다.

toItem은 kind가 "like"도 "comment"도 아니면 그 행을 목록에서 뺀다. read는 readAt !== null이다.

헤더 버튼의 문구는 알림이다. 로그인하지 않으면 NotificationMenu는 null을 반환한다.

참고: MDN Promise.all, Next.js 서버 액션.

4. 토스트가 뜨는 시점

seenRef는 Set이다. 처음 apply가 호출되면 이 Set이 null이라, 받아 온 id만 넣고 토스트는 만들지 않는다. 그래서 새로고침 직후의 기존 알림은 배지로만 남는다.

그 다음 조회에서 읽지 않았고 Set에 없는 id가 있으면 토스트를 앞에 붙인다. 화면에는 최대 3개다. 토스트가 하나라도 있으면 4초 뒤 `setTimeout`이 배열의 마지막 항목을 하나 뺀다. 새로 붙인 항목은 앞쪽이므로, 먼저 온 토스트가 뒤에서 빠진다.

문구는 두 가지다. 좋아요는 `님이 내 글에 좋아요를 눌렀습니다.`, 그 외 `kind`는 `님이 내 글에 댓글을 남겼습니다.`

토스트나 목록 항목을 누르면 `FormData`에 `notificationId`를 넣고 `markNotificationRead`를 호출한 뒤 `/postdetail/[postId]`로 이동한다. `모두 읽음`은 `markAllNotificationsRead`를 호출한다. 두 액션 모두 세션 사용자가 없거나, 단건 읽음에서 id가 그 사용자의 행이 아니면 데이터베이스를 바꾸지 않는다.

5. 로그인 상태가 바뀔 때

`isLoggedIn`이 바뀌면 effect 안에서 `setSnapshot`을 호출하지 않는다. 렌더 중에 `trackedLogin !== isLoggedIn`이면 `trackedLogin`, 목록, 토스트, 패널을 그 자리에서 맞춘다. React가 안내하는 방식은, props가 바뀌었을 때 이전 렌더의 상태를 렌더 중에 고치는 것이다. effect 본문에서 동기적으로 `setState`를 호출하면 렌더가 한 번 더 이어진다.

effect는 로그아웃일 때 `seenRef.current = null`만 하고 반환한다. ref를 지우는 일은 렌더 밖이다. 로그인일 때만 간격 조회를 구독한다.

참고: React, effect 없이 상태 맞추기, React useEffect, MDN FormData.

6. 이 코드가 하지 않는 일

- 브라우저 알림 권한이나 서비스 워커 푸시로 이 행을 보여 주지 않는다.
- 알림 목록을 `"use cache"`에 넣지 않는다. 5초마다 서버 액션이 데이터베이스를 읽는다.
- 댓글 삭제 알림은 없다. 이 앱의 댓글 삭제가 이 함수와 연결되어 있지 않다.